

Sistema Nacional

de Gestión de Recursos y Riesgos

Guía de inicio - Entrega 3

Node 24 + Express 5 + TypeScript + Prisma 7 + PostgreSQL

Estado: en desarrollo - login y seed verificados

Subcarpeta `sistema_nacional/` del repositorio `rescuesync`

Objetivo Que cualquier integrante del equipo pueda hacer git pull, levantar la API del Sistema Nacional en su PC, cargar los datos iniciales e iniciar sesión para obtener un token JWT.

Flujo general

```
Bonita (motor BPM) -----\
                                >--- JWT (rol ADMIN) ---> Sistema Nacional (Express, :3001)
Backend local (Node, :3000) -/                                |
                                                                v
                                                                PostgreSQL (schema sistema_nacional)
                                                                mismo contenedor que la app web
```

8 de octubre de 2026

1. Estado actual del sistema

La API es la “API del cloud” que pide el enunciado: JWT, dockerizada, con Swagger y desplegable en Render. Hoy están listos el esqueleto, el modelo de datos y la autenticación básica.

Ya funciona

- GET /health: estado del servicio.
- POST /login: devuelve un JWT (8 horas) con *sub*, *rol* y *ongId*.
- Middleware `requireAuth(...roles)` para proteger rutas por rol (aún sin rutas que lo usen).
- Modelo de datos y migraciones aplicables con Prisma.
- Seed repetible: cuenta ADMIN, 5 tipos de recurso y 6 certificaciones.

Pendiente (en orden sugerido)

- POST /register público y aprobación de ONGs por el ADMIN.
- Catálogo: certificaciones, ONGs habilitadas y disponibilidad global de recursos.
- Validación por niveles de competencia (devuelve capacidades y riesgo permitido, sin rechazos binarios).
- Bloqueo transaccional de recursos con locking en PostgreSQL y test de concurrencia.
- Liberación y cierre, Swagger, Dockerfile y deploy en Render.

Pendiente intencional Todavía no hay endpoints de negocio: el motor de Bonita y el backend local aún no pueden invocar validación, bloqueo ni liberación. Consorcios y ofertas parciales quedan fuera: viven en la base local de la app web.

Qué viene con git pull y qué es local

Componente	¿Viene por Git?	Observación
Código (src/), schema y migraciones	Sí	Incluye las migraciones init y rol_admin.
package.json y package-lock.json	Sí	Instalar con npm ci.
.env.example	Sí	Plantilla de variables.
.env	No	Se crea desde .env.example.
node_modules/ y src/generated/	No	Se obtienen con npm ci y npm run db:generate.
PostgreSQL y sus datos	No	Cada PC levanta el contenedor y aplica migraciones.
Usuario ADMIN y catálogo	No	Los crea npm run db:seed.

2. Requisitos

- Git
- Node.js 24.x y npm (el repo trae `.nvmrc` con 24)
- Docker Desktop, para el PostgreSQL del repositorio

```
node --version
docker --version
```

Actualizar el repositorio

```
cd RUTA/rescuesync
git status
git pull origin master
```

3. Levantar PostgreSQL

El Sistema Nacional **no tiene su propio contenedor**: usa el Postgres del repositorio raíz (rescuesync-postgres, puerto 5432), con su propio *schema* llamado `sistema_nacional` dentro de la base rescuesync. Las tablas de la app web quedan en el schema `public` y no se mezclan.

```
# desde la carpeta raíz del repo
docker compose up -d
docker compose ps
```

Referencia Si el contenedor ya está corriendo por la app web, no hace falta hacer nada más: se comparte. El schema se crea solo al aplicar las migraciones.

4. Instalar, migrar y cargar datos

```
cd sistema_nacional
npm ci
cp .env.example .env          # PowerShell: Copy-Item .env.example .env
```

Variables de `.env` (los valores del ejemplo sirven para desarrollo local):

Variable	Ejemplo	Para qué sirve
PORT	3001	Puerto de la API. No usar 3000: lo ocupa el backend local.
DATABASE_URL	postgresql://rescuesync:rescuesync@localhost:5432/rescuesync?schema=sistema_nacional	Conexión y schema. El <code>?schema=</code> es obligatorio.
JWT_SECRET	dev-secret-cambiar-en-produccion	Clave de firma de los tokens. Obligatoria.
ADMIN_EMAIL	admin@rescuesync.local	Email de la cuenta ADMIN que crea el seed.
ADMIN_PASSWORD	admin1234	Contraseña del ADMIN (solo desarrollo).

```

npm run db:generate      # genera el cliente de Prisma (src/generated)
npx prisma migrate deploy # aplica las migraciones al schema sistema_nacional
npm run db:seed          # ADMIN, tipos de recurso y certificaciones

```

Nota El seed es repetible: se puede correr varias veces sin duplicar datos. `npm run db:migrate` (`prisma migrate dev`) es solo para crear migraciones nuevas durante el desarrollo; para una instalación limpia usar `migrate deploy`.

5. Arrancar y probar

```

npm run dev              # tsx watch, recarga al guardar

```

En otra terminal, verificar que responde y obtener un token:

```

curl http://localhost:3001/health
# {"status":"ok"}

curl -X POST http://localhost:3001/login \
  -H "Content-Type: application/json" \
  -d '{"email":"admin@rescuesync.local","password":"admin1234"}'
# {"token":"eyJ...", "tokenType":"Bearer", "expiresIn":28800, "rol":"ADMIN", "ongId":null}

```

Respuestas esperadas de /login

Caso	Código	Respuesta
Credenciales correctas	200	token, tokenType, expiresIn, rol, ongId
Contraseña errónea o email inexistente	401	Credenciales inválidas (mismo mensaje en ambos casos)
Falta email o password	400	email y password son obligatorios

Para llamar a rutas protegidas se envía el header `Authorization: Bearer <token>`.

6. Roles y autenticación

Rol	Quién lo usa	Qué puede hacer
ADMIN	Cuenta de servicio de la plataforma RescueSync: el motor de Bonita y el backend local	Validar, bloquear, liberar recursos y aprobar ONGs. El ongId viaja en el body, no en el token.
ONG	Cada ONG registrada, con su propio usuario	Solo lectura: su perfil, sus certificaciones y el catálogo.

Importante ADMIN es una credencial compartida con mucho poder. En producción debe tener una contraseña propia y un `JWT_SECRET` distinto al de desarrollo, definidos como variables de entorno en Render y nunca en el repositorio.

7. Modelo de datos

Tabla	Contenido
Usuario	email, passwordHash, rol (ADMIN u ONG) y ongId opcional y único.

Tabla	Contenido
Ong	nombre, cuil único y estado (PENDIENTE, HABILITADA o SUSPENDIDA).
TipoRecurso	Tipos de recurso con su unidad (PARAMEDICO, RACION_ALIMENTO, AGUA_POTABLE, KIT_MEDICO, VEHICULO_RESCATE).
Certificacion / OngCertificacion	Cada certificación habilita un tipo de recurso hasta un nivel de riesgo máximo; la ONG la tiene con fecha de vencimiento.
OngRecurso	total y bloqueado por ONG y tipo. Un CHECK impide que bloqueado salga de 0 a total.
Compromiso / CompromisoItem	Reserva de recursos por emergencia, lote y ONG. Es único por (emergenciald, loteld, onglid) para que los reintentos no dupliquen.

8. Scripts disponibles

Comando	Qué hace
npm run dev	Arranca la API con recarga automática.
npm run build	Genera el cliente de Prisma y compila a dist/.
npm start	Ejecuta la versión compilada (node dist/server.js).
npm run typecheck	Verifica los tipos sin compilar.
npm run db:generate	Regenera el cliente de Prisma.
npm run db:migrate	Crea y aplica una migración nueva (solo desarrollo).
npm run db:seed	Carga ADMIN, tipos de recurso y certificaciones.

9. Problemas frecuentes

Síntoma	Causa y solución
Falta JWT_SECRET al arrancar	No existe .env o le falta la variable. Copiar .env.example.
Falta ADMIN_PASSWORD al correr el seed	Completar ADMIN_PASSWORD en .env.
Cannot find module ./generated/prisma/client	Falta el cliente de Prisma: correr npm run db:generate.
EADDRINUSE en el puerto	Otro proceso usa el puerto. Verificar que PORT sea 3001 y no haya otra instancia corriendo.
relation "Usuario" does not exist	Faltan migraciones o el ?schema=sistema_nacional no está en DATABASE_URL: correr npx prisma migrate deploy.
ECONNREFUSED a la base	El contenedor no está levantado: docker compose up -d en la raíz del repo.
Versión de Node incorrecta	Usar Node 24 (nvm use, que lee .nvmrc).

10. Notas para el despliegue en Render

- Root Directory y Build Filter apuntan a la subcarpeta `sistema_nacional`.
- El PostgreSQL de Render es aparte; su `DATABASE_URL` no lleva `?schema=` y usa `public`.

- El build debe ejecutar `prisma generate` (ya incluido en `npm run build`) y el arranque debe aplicar `prisma migrate deploy`.
- *prisma* está en `devDependencies`: instalarlo en el build o moverlo a `dependencies` si el arranque corre migraciones.
- Definir `JWT_SECRET`, `ADMIN_EMAIL` y `ADMIN_PASSWORD` como variables de entorno propias.